Input Space Partitioning

Members:

Dhruv Patel

Vraj Patel

Contents

1. Introduction 2

2. Specification of the Program Under Test 2

3. Input Space Partitioning 5

4. Combinatorial Test Generation using ACTS 7

6. Results and Analysis 13

7. Reflection 16

1. Introduction

In this assignment, we aimed to evaluate the csvtojson tool for its effectiveness, using Input Space Partitioning (ISP) and combinatorial testing. Using software testing methods, our evaluation will focus on the tool's accuracy, performance, and usability.

csvtojson is a popular open-source library and command-line tool designed to convert CSV files into JSON format efficiently. It offers various features such as customizable parsing options, asynchronous processing, and integration with data. We decided to choose this tool due to its wide range of configurable parameters that make it suitable for in-depth testing.

Our primary goal is to evaluate the tool in a detailed and careful manner through a structured testing approach. By evaluating through input space partitioning and combinatorial testing techniques, specifically utilizing the ACTS (Automated Combinatorial Testing for Software) framework, we want to ensure comprehensive coverage of various input scenarios while maintaining efficiency in our test case generation.

Our report will detail the specifications of the csvtojson tool, describe our testing methodology, and present the results of our tests with an analysis of any failed cases observed. These findings will provide us with an in-depth insight into the tool's performance and reliability.

2. Specification of the Program Under Test

As mentioned in our introduction, the tool we selected, csvtojson, is an open-source Node.js library and command-line tool that convert CSV files to JSON format. The installation guidelines of the tool have been mentioned in our readMe file which is included in our Test Infrastructure. The tool produces an output based on various parameters for parsing CSV files and offers flexibility in handling different file structures and data types. The JSON format transforms each row in the CSV into a JSON object, resulting in either a JSON array or a series of JSON objects. Corner cases, such as missing fields, empty rows, or special characters in fields, can impact how the program transforms CSV to JSON. Hence, we decided to choose 18 parameters out of several more to ensure proper conversion during the testing phase.

2.1 Input Requirements

The tool reads in CSV files and processes them based on several parameters which strictly follow CSV definition RF4180. RFC 4180 defines the common format and type for CSV files, ensuring a standardized way to represent tabular data in plain text. This includes how fields are separated by commas and how values are quotes when they contain special characters. Additionally, the tools support various command-line parameters.

Below is a concise summary of the 18 parameters we have decided to choose for testing:

Parameter	Purpose	Supported data
output	The format to which the CSV is converted.	json, csv, line
delimiter	Delimiter used to separate columns.	Comma, Tab, Semicolon, Pipe, Auto
quote	Quote character used to surround column content.	DoubleQuote, SingleQuote, Off
trim	Trim spaces surrounding column content.	True, False
checkType	Whether to check the field type.	True, False
ignoreEmpty	Ignore empty values in CSV columns.	True, False
noheader	Indicates whether the CSV has no header row.	True, False
headers	Custom headers for CSV data.	Default, Custom
flatKeys	Don't interpret dots or brackets in headers as nested identifiers.	True, False
maxRowLength	Maximum number of characters in a row.	0 (no limit), 65535
checkColumn	Check if the number of columns matches the number of headers.	True, False
eol	End-of-line character.	LF, CRLF
escape	Escape character for quoted columns.	DoubleQuote, Backslash
includeColumns	Specify which columns to include.	All, NameAndAge, LocationFields
ignoreColumns	Specify which columns to ignore.	None, NameField, LocationFields
alwaysSplitAtEOL	Force each line to be interpreted as a row.	True, False
nullObject	Whether to interpret "null" as a string or a null object.	True, False
downstreamFormat	JSON array format to be output.	line, array

All the parameters that we chose for testing are the ones that are supported by the tool (Please read readMe page of csvtojson for further clarification). Using the predefined parameters allowed us to test the tool rigorously. Introducing new parameters would complicate the process,

as some might not be supported by the tool. However, such instances were tackled by use of constraints (more in section 3).

2.2 Output Requirements

For JSON output, the csvtojson tool follows the JSON standard as defined by ECMA-404 and RFC 8259. These standards specify the syntax and structure for JSON data, ensuring it is a lightweight, text-based, and language-independent format for data interchange.

The table below primarily shows three formats in which the output is specified:

Output Format	**Description**
json	Convert the CSV file into a JSON array
line-by-line json	Convert the CSV file into a line-by-line JSON string
csv	Output the CSV file as it is, but with any specifications given

It is important to note that during our testing, we found out that several parameters such as delimiter, headers, includeColumns and ignoreColumns influenced the formatting of the JSON file. For example, The delimiter determines how column names are separated in the CSV file, which is reflected in the JSON output, and if no headers are provided, the column names in the JSON file will be generic (e.g., name1, name2, …).

2.3 Corner Cases

During testing, we considered corner cases (detailed in section 3) to identify unusual edge scenarios where the input affects the output's correctness. This was due to non-standard CSV formatting and uncommon data structures, which we addressed using a post-processing step in our test infrastructure (explained in section 5).

Some of the corner cases which we considered and tested are (exact cases which were part of the testing are outlined in section 3 under constraints):

- Empty CSV files. In case this case, JSON file will be empty.
- Inconsistent number of columns across rows. For example, very rare but different rows can not have different number of columns.
- Extremely long rows or large input files. For example, If the maxRowLength parameter is set, exceeding this limit should output an error.

- Quoted fields containing delimiters or newlines. In correct formatting can lead to mismatched data in JSON output.
- Mixed data strings (strings, characters, integers, etc). We need to keep in mind the output is consistent. In our testing, we noticed, there were cases where integers were represented as strings.
- Missing headers. As mentioned before, generic field names can cause confusion.

2.4 Input-Output Relationship

The input CSV file's structure directly affects the format of the output. As we mentioned in some cases in section 2.3, how the csv is formatted is extremely important to get the right output. Based on some of the parameters, we did evaluate how the input and output relate:

- The delimiter parameter if not formatted properly, the separate and independent regions affect the JSON objects.
- The quote style, where double or single impacts the parsing of certain components.
- There are instances where certain values are empty (null) in the CSV file. However, the tool omits the columns in the JSON file resulting in a failed test case (ignoreEmpty parameter)
- An important aspect is also how leading whitespace can impact the output.
- Several other parameters such as escape characters, line ending, nested structures and header handling need to be structured properly. Otherwise, it will influence the JSON output.

3. Input Space Partitioning

For the csvtojson tool, we identified several input characteristics from both the CSV file and the command-line parameters that could influence how the file is parsed and converted. To ensure a detailed test coverage, we partitioned the input space based on these characteristics resulting in a **functionality-based input domain model**.

CSV File Characteristics:

Characteristic	Partition 1	Partition 2	Partition 3	Constraints
Rows	0 rows	1 row	Several rows	
Columns	0 columns	1 column	Several columns	Only works with more than 1 column
Data Types	All strings/character	All integers	Mix types	

Missing Values	No missing values	Some missing values		
Field Delimeter	Comma	Tab	Semicolon, Pipe, Automatic	Pipe delimiter requires quoting
Line Endings	LF (Unix-style)	CRLF (Windows-style)		

Command-Line Parameters:

Characteristics	Partition 1	Partition 2	Partition 3	Constraints
Output Format	JSON	CSV	Line-by-line JSON	
Trimming	Enabled	Disabled		checking the type does require trimming
Type Checking	Enabled	Disabled		data types need to be checked
Headers	Default	Custom	No headers	no header requires default headers
Max Row Length	No limit	Restricted (65535)		long rows need to be reduced
Escape Character	Double quotes	Single quotes	No quotes	
Check Column	Enabled	Disabled		header names need to be checked
Flat keys	Enabled	Disabled		
Include Columns	Include All Columns	Include Specific Columns	Exclude specific columns	specific columns require column checking
Ignore Columns	Ignore None	Ignore Specific		

Empty Values	Ignore empty	Keep empty		
Always Split at EOL	Always	Never		
Null Object	Actual null (True)	As a string (False)		
Format	Array	Line-by-Line		

JSON Output Characteristics:

Characteristics	Partition 1	Partition 2	Partition 3	Constraints
Structure	Array of objects	Nested objects	Line-by-line JSON	If the JSON structure is flat, the CSV cannot contain nested fields.
Null values	Null as empty string	Null as null		When null object is true, empty values are represented as JSON null
Field Names	From headers	Custom field names	Generic	If no headers are provided, field names are generated automatically.

4. Combinatorial Test Generation using ACTS

To cover the wide range of input combinations for the csvtojson too, we used the ACTS tool (Advanced Combinatorial Testing System) for combinatorial test generation. The tool enabled us to create test cases between several input parameters without requiring to test every possible combination. Every possible combination would be almost impossible to test manually on an open-source tool that requires a detailed understanding. For this assignment, we focused on **pairwise testing** to ensure each characteristic is tested at least once.

4.1 ACTS Setup

We started by creating a scratch project defining the input domain model using the characteristics identified during the input space partitioning phase. We used 18 parameters (as mentioned in section 2). Each parameter was assigned appropriate values based on the csvtojson tool's configuration options. Each parameter was divided into CSV file characteristics, command-line parameters, and JSON output characteristics. These parameters were included into ACTS framework using enums, booleans, and integers to represent the possible values for each characteristic.

There were other parameters defined in the readMe file that we decided not include in the ACTS configuration. Below are the omitted parameters:

- **fork (experimental):** This allows the process to be forked for concurrent parsing of large CSV files. This parameter is marked as experimental. Hence, it is likely not stable or widely used. Including this may have made it harder to test or cause unpredictable behavior during testing.
- **colParser**: This allows custom parsing logic for specific columns. It accepts a JSON object to specify how individual columns should be parsed. We considered this for advanced custom column parsing. It would have been useful for edge cases. However, for simplicity, we decided to not to use it.
- **needEmitAll**: This whether the parser should build the full JSON result at once or emit partial results as they are parsed. This basically controls the **emission of partial results** as the CSV is parsed, which is a performance optimization feature. Since, it would not directly influence the output result, we did not use it.

4.2 Pairwise Test Case Generation

ACTS generated 16 test cases covering all pairwise interactions between the 18 parameters. These test cases represent a significant reduction from the total possible combinations which would have been 3317760 cases otherwise (3x5x3x2^10x3^2x2^3) if each combination was tested.

Below is the list of test cases generated by ACTS (Please review the **csvtojson_testcases.xml** in our Test Infrastructure for a detailed view):

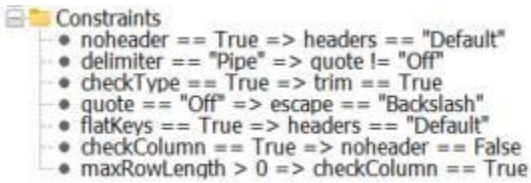
	OUTPUT	DELIMITER	QUOTE	TRIM	CHECKTYPE	IGNOREEMPTY	NOHEADER	HEADERS	FLATKEYS	MAXROWLENGTH	CHECKCOLUMN	EOL	ESCAPE	INCLUDECOLUMNS	IGNORECOLUMNS	ALWAYSSPLITATEOL	NULLOBJECT	DOWNSTREAMFORMAT
1	json	Comma	SingleQuote	false	false	false	false	Custom	false	65535	true	CRLF	Backslash	NameandAge	NameField	false	false	array
2	csv	Comma	Off	true	true	true	true	Default	true	0	false	LF	Backslash	LocationFields	LocationFields	true	true	line
3	line	Comma	DoubleQuote	false	false	true	true	Default	false	0	false	CRLF	DoubleQuote	All	None	false	false	line
4	json	Tab	Off	true	true	false	false	Custom	false	0	true	LF	Backslash	All	NameField	true	true	array
5	csv	Tab	DoubleQuote	false	false	false	true	Default	true	0	false	CRLF	DoubleQuote	NameandAge	LocationFields	true	true	array
6	line	Tab	SingleQuote	true	true	false	false	Custom	false	65535	true	LF	DoubleQuote	LocationFields	None	false	false	line
7	json	Semicolon	DoubleQuote	false	false	true	true	Default	true	0	false	LF	DoubleQuote	LocationFields	NameField	false	true	array
8	csv	Semicolon	SingleQuote	true	true	true	false	Custom	false	65535	true	CRLF	Backslash	All	LocationFields	true	false	line
9	line	Semicolon	Off	false	false	false	false	Default	true	65535	true	LF	Backslash	NameandAge	None	true	true	array
10	json	Pipe	DoubleQuote	true	true	true	false	Default	true	65535	true	CRLF	Backslash	LocationFields	LocationFields	false	false	line
11	csv	Pipe	SingleQuote	false	false	false	true	Default	true	0	false	LF	DoubleQuote	All	None	true	true	array
12	line	Pipe	DoubleQuote	true	true	true	false	Custom	false	0	false	LF	Backslash	NameandAge	NameField	true	false	line
13	json	Auto	Off	false	false	false	false	Custom	false	65535	true	CRLF	Backslash	LocationFields	None	false	false	array
14	csv	Auto	DoubleQuote	true	true	true	true	Default	true	0	false	LF	DoubleQuote	All	NameField	true	true	line
15	line	Auto	SingleQuote	true	false	false	true	Default	true	0	false	CRLF	DoubleQuote	NameandAge	LocationFields	false	true	line
16	csv	Tab	DoubleQuote	true	false	true	true	Default	false	0	false	CRLF	DoubleQuote	LocationFields	LocationFields	false	true	array

With this test case, we created 16 input csv files (1 file per test case), that would meet all the specified requirements for that particular file. For example: **Test Case 1**: Output Format: JSON, Field Delimiter: Comma, Trimming: Enabled, Rows: 0, Columns: 1, Missing Values: No, Escape Character: Double Quotes, etc.

To have a detailed explanation of our test files, have a look at section 5.

4.3 Constraints in ACTS

To ensure valid and meaningful test cases, we implemented several constraints in ACTS. Below is the list of constraints that we picked as the most important as defined in ACTS (Detailed descriptions of these constraints are given in section 3).



These constraints have been described in section 3. To avoid repetition, I will not mention it again.

These constraints were encoded into ACTS to ensure that only valid combinations were tested, preventing unnecessary or unrealistic test cases from being generated.

4.4 Test Plan

With the test cases generated and constraints applied, we developed a comprehensive test plan to automate the execution and evaluation of the tests. The test plan involved the following steps:

- Test Data Generation: As mentioned before, 16 test files (CSV) were generated.
- Actual and Expected JSON output files: Actual output files used by the tool csvtojson for conversion, while we programmed an oracle to generate the expected files for comparison.
- Automation of Test Execution: We developed a few python-based test harness scripts to automate the testing process. Detailed explanations of each file will be in section 5.
- Reporting Results: After running the test suite, the test harness produced a report detailing which test cases passed and which failed.
- Analysis: We analyzed the results of the test cases to determine if any issues or bugs were uncovered. In cases where test failures occurred, we examined the specific input conditions and output discrepancies to identify the root cause of the problem. This was done manually on the failed cases. However, for this tool, we only had a few test cases that failed. For extensive testing, we would not suggest this approach.

5. Test Infrastructure

5.1 Directory descriptions

To thoroughly evaluate the csvtojson tool, we developed a comprehensive test infrastructure that automates the process of generating test cases, executing them, and comparing the results. Our code base included the following:

- **TestFiles**: This directory contains all the input CSV files used for testing. Each test case, generated by ACTS, corresponds to an individual CSV file (e.g., testx.csv) that defines the input for a specific test scenario.
- **ActualMessages**: This directory logs **actual messages** (error messages or passed tests) produced by the csvtojson tool during the test execution. Each test case has an associated message file (e.g., testx_message.txt) where any differences between expected and actual messages are recorded. Additionally, comparison_log.txt provides a summary of the comparison results.
- **ActualOutput**: The actual output JSON files generated by running the csvtojson tool on each test input file are stored here. For every test case, a corresponding testx.json file is generated, which will be compared against the expected output to verify correctness.
- **ExpectedMessages**: This directory contains the expected informational messages for each test case (manually formed) which would be useful to test edge cases.
- **ExpectedOutput**: This directory contains the expected JSON output files (e.g., testx.json) that represent the correct output for each corresponding input CSV file in the TestFiles/ directory. These files serve as the reference output, and the actual output from the csvtojson tool will be compared against these files to determine if the test has passed or failed.
- **Scripts**: This directory contains all the **Python scripts** necessary for automating test generation, execution, and comparison. Each file has been explained in detail in section 5.2.

Our infrastructure is organized into several key components as detailed below:

```
TestData
├── ActualMessages
│   ├── comparison_log.txt
│   ├── testx_message.txt
├── ActualOutput
│   └── testx.json
├── ExpectedMessages
│   └── testx_message.txt
├── ExpectedOutput
│   └── testx.json
├── Scripts
│   ├── compare_actual_vs_expected_output.py
│   ├── comparison_log.py
│   ├── actual_messages.py
│   ├── generate_actual_output.py
│   ├── generate_expected_output.py
│   ├── generate_input_files.py
│   ├── post_processing.py
└── TestFiles
    └── testx.csv
```

5.2 Scripts

The scripts in our test infrastructure automate various parts of our testing process, from generating inputs to comparing actual and expected outputs. The reason we chose to automate the process is that for such a large test case, implementing each file would be challenging and might have not been accurate. Below is the list of the files we implemented:

1. generate_input_files.py:
    - This script generates CSV input files based on the test cases defined in our input space partitioning.
    - Our script:
        - We started by defining a dataset in 2D array format where the first row contains the column headers and it follows with individual data. This is a basic input for all test cases.
        - We define a few helper functions to help with following the characteristics we defined in section 3. Some of these include:
            - Trimming whitespace from strings if true (enabled).

        - Convert empty string fields to None if the IgnoreEmpty parameter is set to true (enabled).
        - Include columns and exclude column functions. Based on the characteristics, this will be either set to true or false.
        - Set a limit to 65535 for maxRowLength as seen in the parameter list.
    - We defined a map for a delimiter (comma, tab, semicolon, pipe, and auto) and a quote (double quote, single quote, or none).
    - We defined 16 separate functions for each test case that was generated by ACTS. Each test case specifies the parameters for that particular test. For example, test case 1 has no header in the output and only includes two columns, a delimiter is common, single quotes, and a max row length is applied. Similarly, the other 15 test cases follow their respective parameter characteristics.

2. generate_actual_output.py
    - This script runs the csvtojson tool on each test CSV file and generates the corresponding JSON output files. Basically, it goes through each file in TestFiles/ directory and stores the JSON files in the ActualOutput/ directory.
    - After a lot of research, which we struggled initially, we figured out using import subprocess from python would allow us to run the tool from the script. This was important because csvtojson is a separate executable program that needs to be run as a command, and Python itself cannot directly execute or interact with it as a library.

3. generate_expected_output.py
    - This script generates the **expected** JSON output files by converting the test CSV files using a Python library. This was written as an oracle so we could compare the outputs.
    - Reads the CSV files from the TestFiles/ directory and stores JSON output in ExpectedOutput/ directory.

4. compare_actual_vs_expected_output.py
    - This script compares the actual JSON output (generated by csvtojson) with the expected JSON output.
    - This iterates through JSON files in the ExpectedOutput/ and ActualOutput/ directories and compares the content of each file and reports whether the test passed or failed. The results are printed in the command line then.

5. comparison_log.py
    - The script prints out the differences between expected and actual outputs and stores it as comparison_log.txt under ActualMessages/ directory.

6. actual_messages.py
    - This script generates txt files detailing the differences between expected and actual outputs for each test case. It reads the contents of both files and writes a diff report to the ActualMessages/ directory.

7. post_processing.py (incomplete)
    - Our aim in writing this script was to fix structural issues in JSON files after they were generated. This would handle special characters or nested structures during conversion. However, we were not able to succeed at this task. Running the script did not make any changes. Due to our limited knowledge of JSON files, we were not able to fulfill this task.

6. Results and Analysis

Using the ACTS tool, we generated a total of 16 test cases. These cases were designed to cover an in-depth analysis of the input characteristics for the csvtojson tool, utilizing pairwise testing to reduce the number of tests by a significant amount.

Out of our 16 test cases:

- **10 tests passed**: test1.json, test2.json, test3.json, test6.json, test8.json, test9.json, test10.json, test11.json, test13.json, and test15.json.
- **6 tests failed**: test4.json, test5.json, test7.json, test12.json, test14.json, and test16.json.

We achieved an overall 60% success rate.
The failed test cases showed differences between the **expected output** and the **actual output**. The comparison logs indicate differences in fields such as missing data, format inconsistencies, and improper handling of certain specifications.

To understand why these cases failed, we had to manually look at the files to figure out what had happened exactly. We did analyze the six failed cases in detail. Below are the possible reasons for each failed case and what this implies about the tool:

- **Failed Test Case 1 (test4.json)**: This test case failed due to incorrect removal of columns when no headers were provided. Since the header row was absent, the tool did not know which columns to exclude. Further, the Tab delimiter made it complicated as it struggled

to map the correct items. Hence, since no headers were present, csvtojson may have misinterpreted the column positions

- **Failed Test Case 2 (test5.json)**: The combination of quoting and missing values led to the failure. Some fields were missing or empty. When applying double quoting, it caused inconsistencies.
- **Failed Test Case 3 (test5.json)**: This was likely due to This failure occurred due to issues with nested JSON objects. The tool incorrectly nested some fields, likely due to misinterpretation of the semicolon delimiter.
- **Failed Test Case 4 (test5.json)**: The failure here is from the interaction between Pipe delimiters and quoting. The tool failed to link fields where pipes were used as delimiters, especially when some fields were quoted.
- **Failed Test Case 5 (test5.json)**: This test failed because the auto delimiter selection did not work correctly when columns were excluded. The tool automatically used commas which shifted the data incorrectly.
- **Failed Test Case 6 (test5.json)**: This failure occurred due to improper handling of **special characters** within quoted fields. For example, something like a postal code that has a space in between (i.e V5E 4P1), confused the tool.

Across the failed tests, we noticed that the common issues were handling complex delimiters such as pipe or semicolon, managing quotes, missing headers, and issues with special characters.

Additionally, to also understand the passed test cases to know what worked well, we looked at some of these passed results to understand which characteristics were handled correctly. Below are some of the tests we analyzed:

- Passed Test 1: The tool handled the **default comma delimiter** and **quoted fields** with no headers.
- Passed Test 2: The tools linked the Tab delimiter with present headers and quoted fields producing accurate JSON output.
- Passed Test 3: The tool handled the **semicolons as delimiters** correctly and **quoted all fields**, including those with special characters.
- Passed Test 4: The tool correctly linked the **pipe as delimiters** even when **only one column** was included and **no headers** were present.
- Passed Test 5: The tool's **auto delimiter detection** worked correctly, defaulting to the expected comma delimiter.
- Passed Test 6: The tool managed **automatic delimiter detection**, **quoted fields**, and **missing headers** together.
- Passed test 6 to 10 had similar cases.

Across the passed tests, the tool does great work in standard delimiters such as commas and tabs. It also works well with semicolons and pipe if the data is simple. In simple datasets, the tools use quotes well. Additionally, it does handle the data well when there are no headers.

Summary:

- As we mentioned before, 3317760 cases would need to be taken into account if we did not do pairwise testing which is impossible to do manually. Even with an automated script, these would take too long for it to generate. With pairwise testing, only 16 testing were needed.
- As seen, out of 16 cases, we managed to pass 10 tests which is significantly a good number.

6.2 TradeOffs made using Pairwise Testing

Pros:

- Pairwise testing significantly reduced the number of test cases while still ensuring full coverage of between parameters.
    - e.g. Most test cases combined multiple parameters in a single test.
- Pairwise testing ensured that every possible pair of parameter values was tested at least once.
    - e.g. Test case 7 used a semicolon delimiter with double quotes, while test case 8 used a semicolon with single quotes. This pair of tests covered both quote styles with the semicolon delimiter.
- With fewer test cases, we could execute the entire test suite more quickly.

Cons:

- While all pairs of parameters were tested, some specific combinations of three or more parameters were not covered.
    - e.g. In test case 9, we tested the combination of semicolon delimiter with custom headers, but we might have missed testing this with specific quote styles or trimming options.
- Some edge cases that require specific combinations of multiple parameters might not be included in the test suite.
    - e.g While we tested various combinations of delimiters and quote styles, we might not have covered all possible interactions with the ignoreEmpty or checkType options.
- The high efficiency of pairwise testing might lead to overconfidence in the test coverage.
    - The tool may still have bugs that are hard to debug.

7. Reflection

7.1 Challenges we faced

- With 18 parameters, each with multiple possible values, understanding and using all possible interactions was challenging. We had to carefully consider which combinations were most important to test.
- Coding the expected output script to serve as an oracle was challenging, as we needed to replicate the expected behavior of csvtojson.
- We struggled with analyzing each failed output. Interpreting the result was a significant challenge.

7.2 Our experience

- We found that pairwise testing significantly reduced the number of test cases while still providing good coverage. This made the testing process more manageable and efficient.
- The scripts we developed for generating inputs, expected outputs, and comparing results were crucial in making the testing process repeatable and efficient.
- Using ACTS forced us to think systematically about the input space and parameter interactions, leading to a more comprehensive test suite.

7.3 Our learning

- We gained practical experience in applying pairwise testing and using tools like ACTS for test case generation.
- We learned how to write test hardness scripts for easy processing.
- Creating an oracle taught us about the challenges and importance of having a reliable reference for expected behavior.
- We learned how to effectively use constraints in ACTS to ensure meaningful test cases and avoid invalid combinations.
- We applied theoretical concepts like input space partitioning and pairwise testing to a real-world tool, bridging the gap between theory and practice.

ACTS's user-friendly interface and pairwise testing strategy made it straightforward to define input characteristics and constraints which was easy for us to understand. However, dealing with the parameters was extremely difficult. It required a thoughtful and careful process, particularly in planning the entire test infrastructure.

From a technical perspective, having detailed documentation on the tool would have helped in a better understanding of the tool. Additionally, we would be able to come up with a more detailed testing plan to generate additional tests to identify more errors.

References

[1] https://www.npmjs.com/package/csvtojson

[2] https://www.w3schools.com/python/python_json.asp

[3] https://docs.python.org/3/library/csv.html

[4] https://www.cs.cornell.edu/courses/cs5154/2021sp/resources/InputSpacePartitioning.pdf